

Edge Agent. Runtime

Process isolation, hardware fallback,
and fair scheduling for on-device
Indic AI inference.

TL;DR A three-process model — Supervisor, API Server, Inference Worker —

isolates the failure-prone inference path from the always-on local API. The Supervisor keeps a warm worker pool to absorb crashes in under 300 ms. Hardware fallback (Qualcomm NPU → CPU on Windows; ANE → Metal → Accelerate → Cloud on macOS) is gated by a per-device circuit breaker. On the client, a single shell-owned SDK exposes one connection to all MFEs; a weighted-fair scheduler with priority aging enforces the 4-slot capacity contract and prevents starvation during streaming contention.

SUBMITTED BY

[Your Name]

Backend Intern Candidate · Edge Runtime

COMPANION ARTIFACTS

[github.com/\[your-handle\]/sarvam-edge-design](https://github.com/[your-handle]/sarvam-edge-design)

sarvam-edge-demo.vercel.app

1 · Assumptions & System Overview

Stated upfront so a reviewer can challenge any decision by attacking its assumption rather than its conclusion.

— Stated Assumptions

- A1 Model class.** A single 3B–7B parameter Indic LLM is hot-loaded at startup; size on disk $\approx$ 4 GB; resident memory after load $\approx$ 6 GB.

- A2 Hardware tiers.** Windows 11 on x64 (DirectML / CPU) and Snapdragon X ARM64 (Qualcomm NPU); macOS 14+ on Apple Silicon (ANE / Metal / Accelerate).

- A3 API surface.** The Edge Agent exposes `http://127.0.0.1:<port>` only — bound to loopback, never to `0.0.0.0`.

- A4 Clients.** Enterprise web apps, Electron apps, and browser extensions. All are first-party; the enterprise customer controls origins.

- A5 Authentication.** Per-session token issued at agent startup, mirrored to the OS keychain; the shell SDK fetches it via a native helper. The token is bound to the originating user session.

- A6 Concurrency.** Hard cap of 4 concurrent inference requests is a runtime invariant, not a tuning knob, because it derives from KV-cache memory budget.

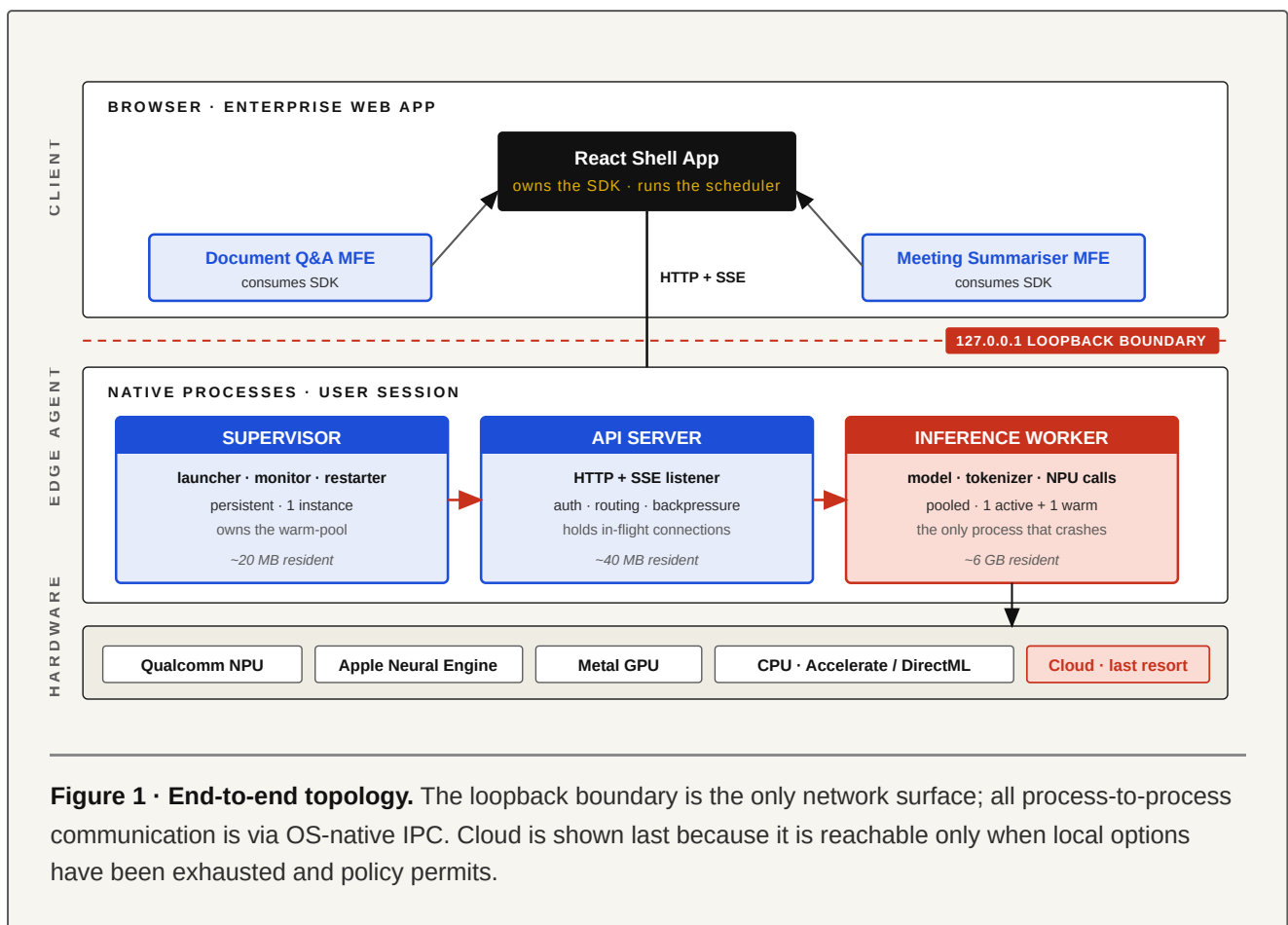
- A7 Streaming transport.** Server-Sent Events over HTTP/1.1; WebSockets considered but rejected (see §B.1 tradeoffs).

- A8 Crash semantics.** The inference worker is the only process expected to crash routinely. Supervisor and API Server are designed to be effectively crash-free; a Supervisor crash is treated as a full agent restart.

- A9 Disk budget.** $\approx$ 50 MB for runtime state (warm-pool snapshot, request journal, crash dumps capped at 5×20 MB).

- A10 Out of scope.** Model download/update, telemetry pipeline, license enforcement, and multi-model hot-swap — flagged for future design.

System Overview



Part A · Runtime Architecture

A.1 · Process Model

The agent spawns **three processes on startup**. The choice of three (not two, not five) is the central architectural call of this design and is justified below.

PROCESS	LIFECYCLE	RESPONSIBILITY	IPC CHANNELS	RESTART POLICY
SUPERVISOR edge-agent	Persistent 1 instance	Spawns and monitors children. Owns the warm-pool. Collects crash dumps. Holds the listening socket (passes FD to API Server). Single source of truth for process state.	spawn() ; heartbeat over named pipe / UDS	If it dies, the whole agent dies. UI installs a launchd / Service watchdog to relaunch.
API SERVER edge-api	Persistent 1 instance	HTTP/SSE listener on 127.0.0.1 . Origin and token validation. Per-request routing into the worker IPC. Holds in-flight client connections during worker restarts.	Inbound HTTP+SSE; outbound to Supervisor (status) and Worker (jobs).	Supervised restart with 1-2-4-8s backoff; clients see 503 with Retry-After .
INFERENCE WORKER edge-worker	Pooled 1 active + 1 warm	Loads model weights, owns KV-cache, calls NPU / ANE / GPU / CPU backends. The only process allowed to fault.	Job channel (length-prefixed protobuf over named pipe / UDS).	Warm replacement on crash; cold restart only if warm pool is exhausted.

WHY THREE, NOT TWO

A two-process model (Supervisor + combined API/Worker) fails the assignment's isolation requirement — a worker crash would tear down the API server and all in-flight client connections.

WHY THREE, NOT FIVE

A separate process per hardware backend (NPU, GPU, CPU) was rejected; the cost of three idle 6 GB workers (≈ 18 GB resident) exceeds the entire memory budget of most target machines. Backend selection happens inside the single worker via a runtime-loaded shim.

Process Lifecycle

COLD-START LIFECYCLE

From user login or service start to first-byte-ready.

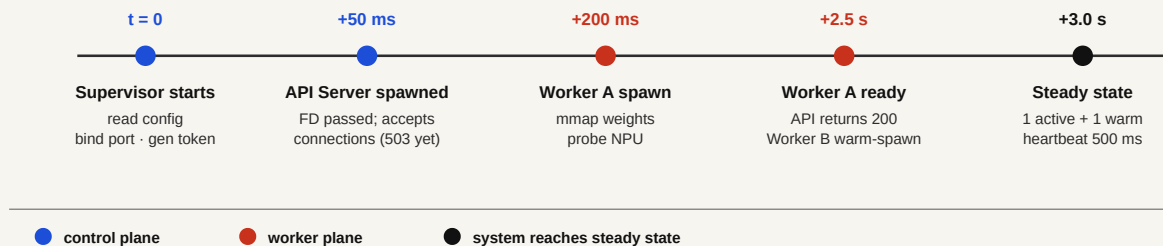
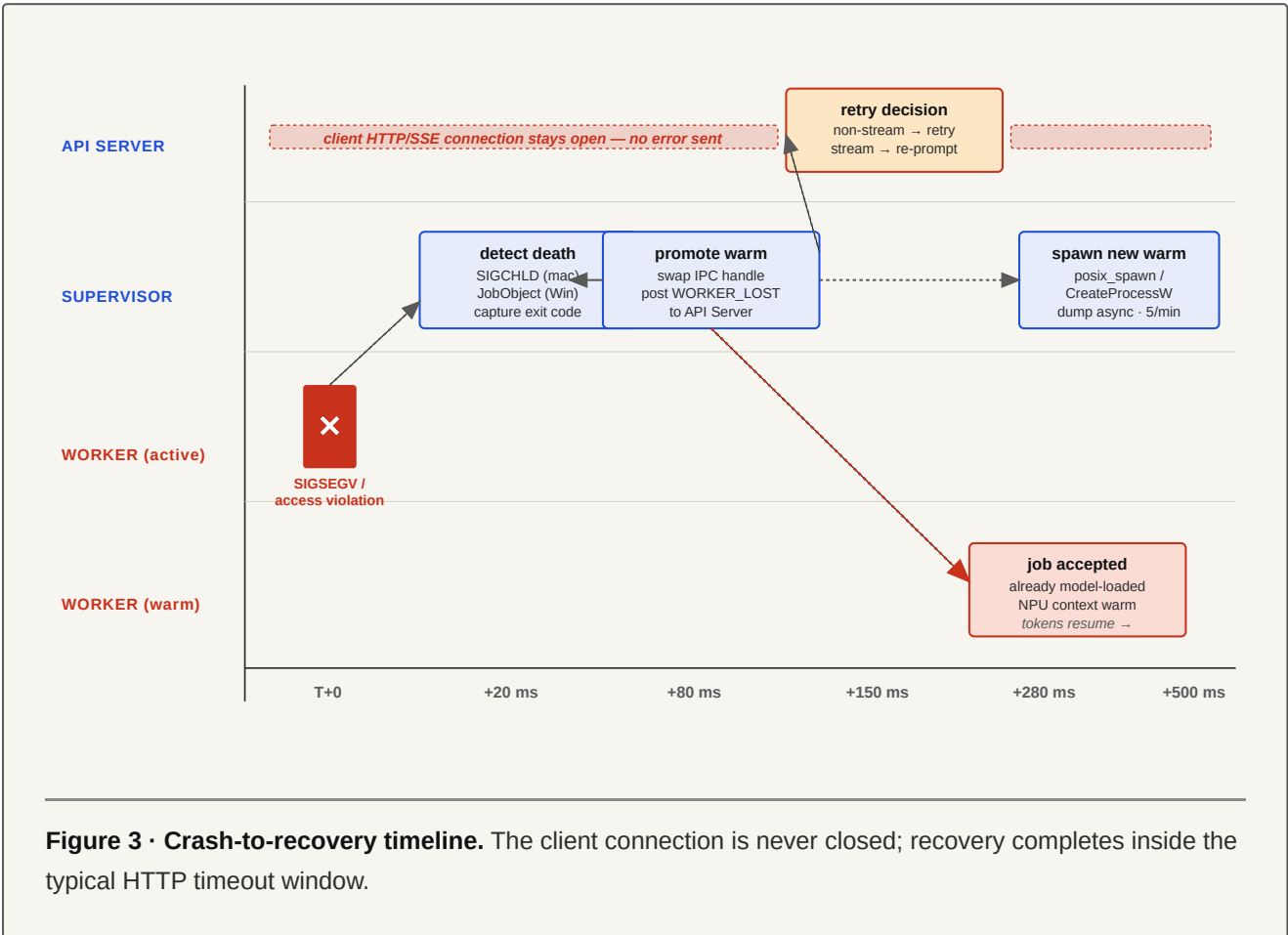


Figure 2 · Cold-start sequence. The warm-pool is established *before* steady state, so the first crash recovery never pays a cold-start cost.

A.2 · Worker Crash Recovery

A Windows access violation kills the active worker mid-request. The Supervisor's job is to make this invisible — or near-invisible — to the client.



WITHIN THE FIRST 500 MS — THE SUPERVISOR:

TIME	ACTION
T + 0	Worker terminates with <code>STATUS_ACCESS_VIOLATION</code> (Windows) or <code>SIGSEGV</code> (mac).
T + 20 ms	Supervisor receives notification via <code>JobObject</code> completion port (Win) or <code>kqueue(EVFILT_PROC)</code> / <code>waitpid</code> (mac). Exit reason captured.
T + 80 ms	Supervisor flips warm worker to active by re-pointing the IPC handle. Sends <code>WORKER_LOST</code> with the old job-IDs to the API Server.
T + 150 ms	API Server consults the per-request retry policy (see below) and either re-dispatches to the new active worker or surfaces an error event over SSE.

T + 280 ms	New worker, already model-loaded, accepts the replayed job. Token generation resumes.
T + 500 ms	Supervisor begins spawning a replacement warm worker in the background. Crash dump written to <code>%LOCALAPPDATA%\Sarvam\Edge\crashes\</code> , rate-limited to 5/min to avoid disk fill from crash loops.

HOW THE API SERVER RESPONDS TO THE IN-FLIGHT REQUEST

Three cases, decided by request type:

REQUEST TYPE	BEHAVIOUR
Non-streaming, idempotent (e.g. classification, embedding)	Auto-replay against new worker. Client never sees an error; latency budget $\pm$ ~280 ms.
Streaming, no tokens emitted yet	Replay from the original prompt. Client receives a single <code>retry</code> SSE event, then normal tokens.
Streaming, tokens partially emitted	Emit <code>error: { code: "WORKER_CRASHED_MID_STREAM", recoverable: true, resume_token: <hash> }</code> . Client may call <code>POST /v1/inference/resume</code> with the prompt + tokens-so-far + <code>resume_token</code> . KV-cache is gone; this is a re-prefill from a longer prompt, not a true resume — communicated honestly to the client.

STATE PERSISTED TO DISK

Three things, all atomic via write-temp-then-rename:

- **Request journal** — last 64 in-flight jobs (prompt hash, request ID, MFE, start time) in `journal.bin`. Used to reason about cascading crashes and for crash-dump correlation. Not used to *replay* across full agent restarts — the API client already does that.
- **Warm-pool descriptor** — backend selection, port number, token. Lets a Supervisor restart (rare) skip rediscovery.
- **Crash dump** — minidump via `MiniDumpWriteDump` (Win) or Apple's `ReportCrash` hook (mac), with the process's own stderr tail prepended.

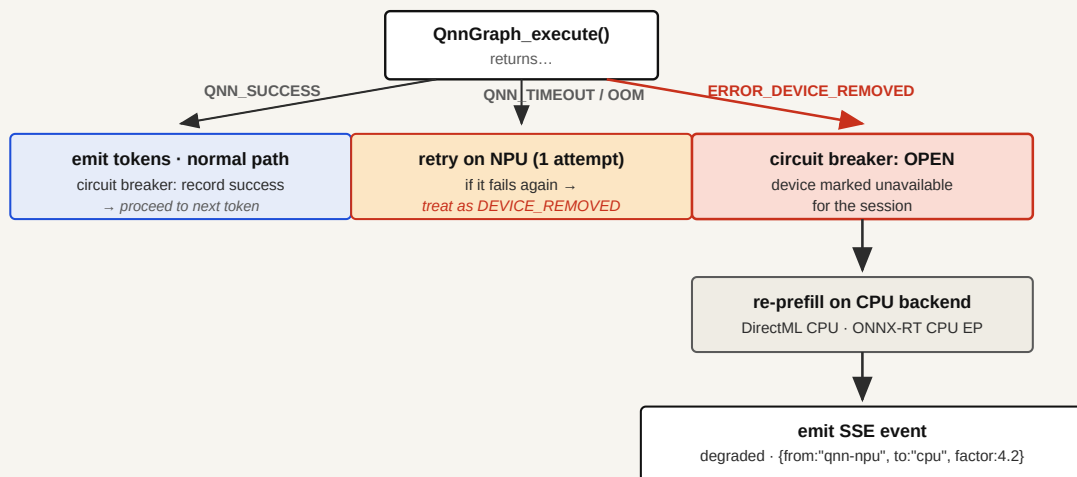
WHY FASTER THAN COLD START A cold worker pays for (1) process creation $\approx$ 50 ms, (2) mmap-ing 4 GB weights and faulting in pages $\approx$ 1.5–2.5 s, (3) NPU context creation and graph

compilation $\approx 300\text{--}800$ ms — total $\approx$ **2–3 seconds**. The warm worker has already paid (2) and (3), so recovery collapses to roughly the IPC switch cost — $\approx$ **280 ms**. That's the entire reason for the warm pool.

A.3 · Qualcomm NPU Failure Handling (Windows ARM64)

When the Hexagon driver returns `ERROR_DEVICE_REMOVED`, the device is dead for the session. The runtime must (a) finish the current request on CPU without a visible failure, (b) prevent the next 100 requests from hitting the same dead device, (c) let recovery happen if and only if a real driver health check passes.

DECISION TREE · NPU CALL OUTCOME



PER-DEVICE CIRCUIT BREAKER · STATE MACHINE

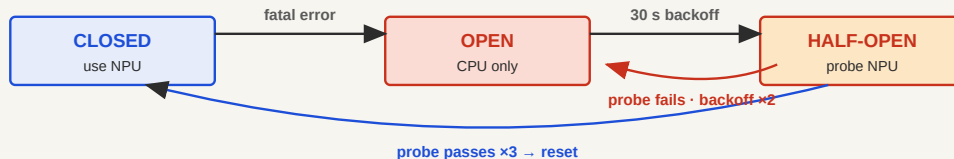


Figure 4 · NPU fallback decision tree (top) and **per-device circuit breaker state machine** (bottom). Probe uses an 8-token prefill on a dedicated probe-graph (~60 ms budget) and must return a known fixed tensor. Backoff doubles each failed probe; cap at 30 minutes.

HOW LATENCY DEGRADATION IS COMMUNICATED TO THE CLIENT

The API contract is preserved — same endpoint, same response schema. Degradation is surfaced as an additional SSE event the client may choose to display:

event: degraded

```
data: {
  "request_id": "req_8f2a...",
  "from_backend": "qnn-npu",
  "to_backend": "cpu-onnxrt",
  "expected_latency_factor": 4.2,
```

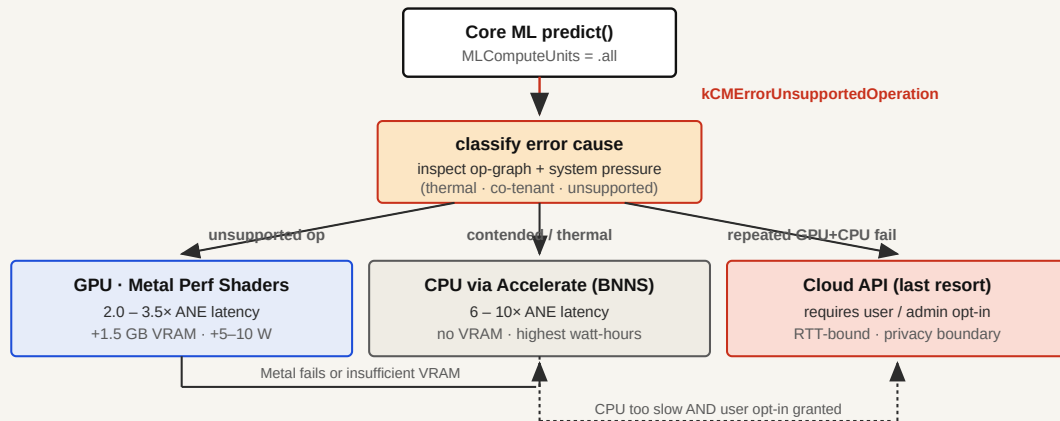
```
"reason": "ERROR_DEVICE_REMOVED",  
"session_scoped": true  
}
```

The standard `token`, `done`, and `error` events continue unchanged. A response header `X-Edge-Backend: cpu-onnxrt` is also set so non-streaming clients see the same information.

A.4 · Apple Neural Engine Fallback (macOS)

The ANE is shared across the system; `kCMErrorUnsupportedOperation` can mean "op not supported" or "ANE is busy with another app." The fallback ladder must distinguish the two and pick the cheapest viable backend.

FALLBACK LADDER · CORE ML ERROR PATH



SELECTION LOGIC · first match wins

1. If error is "unsupported op" AND model has Metal-compatible export → **Metal**.
2. If ANE contention AND `powerSourceState == .battery` AND charge < 20% → **Accelerate** (lower wattage).
3. Else → **Metal**; fall to **Accelerate** on `MTLCommandBufferStatusError`.
4. Cloud only if admin policy allows, no *X-Data-Residency: on-device*, and CPU SLA breach predicted.

Figure 5 · ANE fallback decision tree, escalation order, and selection logic. Cloud is gated behind explicit opt-in, never automatic.

HOW DEGRADED MODE IS COMMUNICATED WITHOUT CHANGING THE API CONTRACT

Identical mechanism to the NPU case — the API surface is one contract, regardless of backend. The client sees:

event: degraded

```
data: {
  "request_id": "req_3c91...",
  "from_backend": "ane",
  "to_backend": "metal-gpu",
  "expected_latency_factor": 2.4,
  "reason": "kCMErrorUnsupportedOperation",
  "session_scoped": false
}
```

Note `session_scoped: false` here: ANE contention is transient (the co-tenant app finishes), so the circuit breaker uses a **5-second sliding window** rather than session-level lockout. The NPU case used `session_scoped: true` because `ERROR_DEVICE_REMOVED` requires a driver reload.

TRADEOFF

Auto-falling to Cloud would maximize "things work" but breaks the product's core promise of on-device inference. The default is to *fail loudly with a degraded local result* rather than silently exfiltrate data. Enterprises that want Cloud fallback must opt in via MDM policy.

Part B · Micro-Frontend Integration

— B.1 · Agent Integration Layer & API Contract

API CONTRACT — REQUEST, STREAMING RESPONSE, ERROR ENVELOPE

POST /v1/inference · request

```
POST /v1/inference HTTP/1.1
Host: 127.0.0.1:7321
Origin: https://app.enterprise.tld
Authorization: Bearer eyJhbGc... (session token from native helper)
X-MFE-Id: doc-qa@2.4.0
X-Priority: user           ← user | background
X-Request-Id: req_8f2a3c91
X-Idempotency-Key: 7d4f9... ← optional; enables safe retry
Accept: text/event-stream  ← omit for non-streaming JSON
Content-Type: application/json

{ "model": "sarvam-7b-indic", "prompt": "...", "max_tokens": 512,
  "stream": true, "stop": ["\n\n"] }
```

200 OK · streaming response opener

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
X-Edge-Backend: qnn-npu
X-Edge-Worker-Id: w_a91f
Cache-Control: no-store

event: queued
data: {"position": 0, "eta_ms": 0}

event: dispatched
data: {"worker_id": "w_a91f", "backend": "qnn-npu"}

event: token
data: {"text": "नमस्ते", "index": 0}
...
```

error envelope · 4xx / 5xx

```
{ "error": {
  "code": "AGENT_AT_CAPACITY",
```

```
"message": "All inference slots are in use",
"recoverable": true,
"retry_after_ms": 1200,
"request_id": "req_8f2a..." } }
```

STATUS	CODE	MEANING
200	OK	Success (sync or streaming opener).
202	QUEUED	Reserved for future fire-and-forget batch endpoint.
401	BAD_TOKEN	Session token missing or expired; re-fetch via native helper.
409	DUPLICATE_IDEMPOTENCY_KEY	Same key already in flight.
429	AGENT_AT_CAPACITY	Client should respect Retry-After or rely on scheduler queue.
503	WORKER_UNAVAILABLE	Warm pool exhausted (cold restart in progress).
507	DEVICE_UNAVAILABLE_DEGRADED	All accelerators failed; response served on CPU.

SHOULD EACH MFE CONNECT DIRECTLY, OR GO THROUGH THE SHELL?

APPROACH	PROS	CONS
Shell-owned singleton SDK CHOSEN	One auth handshake. One scheduler holds the global view of in-flight requests. MFEs get a uniform API. Easy to swap transport (HTTP ↔ WebTransport).	Shell becomes a coordination chokepoint. MFE crash isolation needs care — handled via try/catch boundaries in the SDK.
Direct MFE → Agent	Maximum MFE autonomy; no cross-MFE coupling.	Each MFE re-implements auth, retry, queueing. No global capacity view → MFEs race for the 4 slots. Multiplies connection count.
Shared worker / SharedWorker	Browser-native singleton; survives single-tab close.	Sub-par browser support across enterprise IE/Edge legacy. Adds a third

process boundary on the client side.

WHY SINGLETON WINS

The Edge Agent enforces 4 concurrent requests globally. Without a single client-side coordinator, two MFEs cannot honor priority or fairness — they just race. The shell-owned SDK turns the constraint into a property the MFEs can rely on.

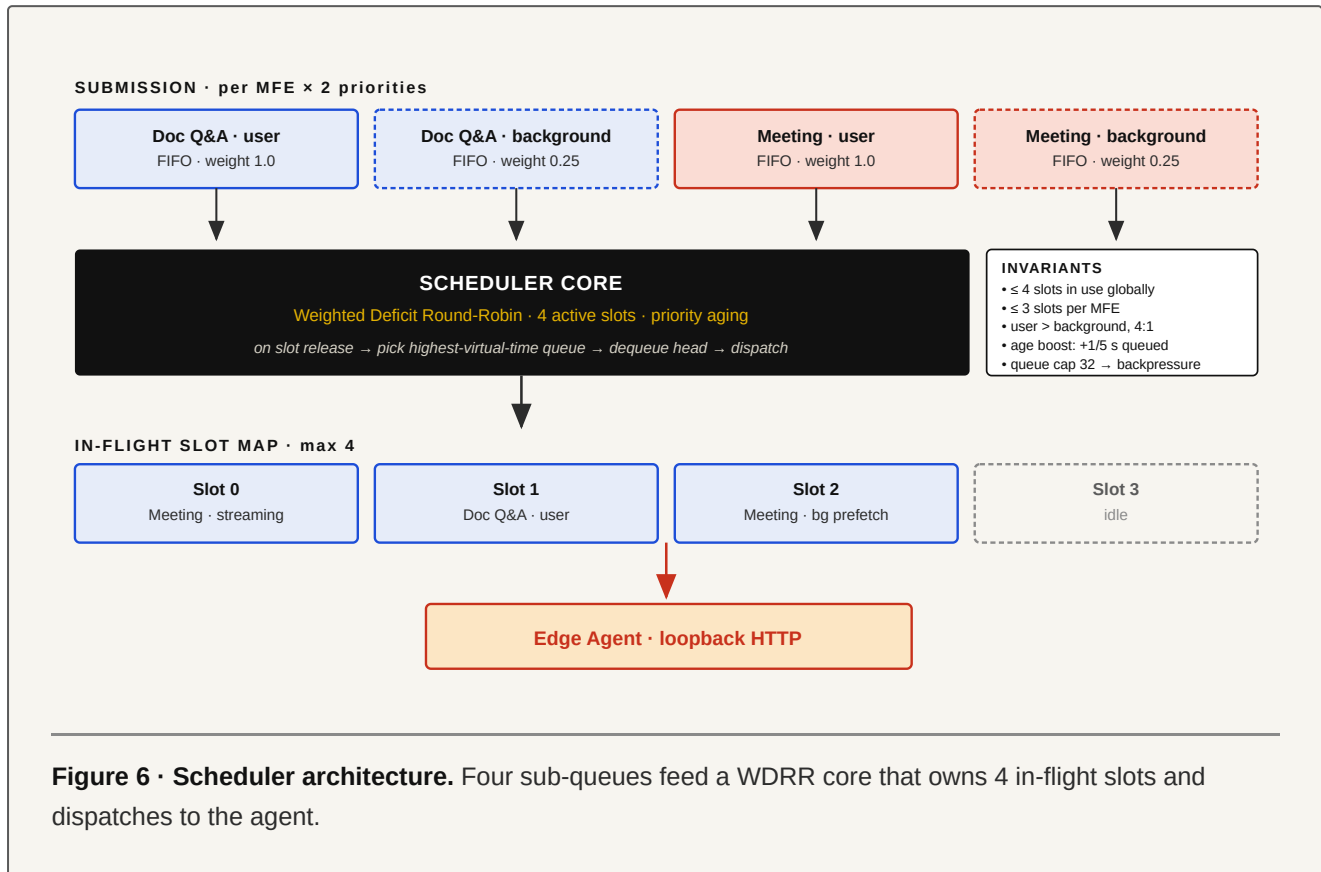
WHY NOT WEBSOCKETS

SSE is purpose-built for server → client streaming, plays nicely with HTTP intermediaries, requires no special agent protocol, and cancels cleanly with AbortController. The bidirectional channel WebSockets provide is unneeded — control messages are short and fit in headers.

Security implications. **Origin pinning** — agent rejects requests whose `Origin` isn't on the MDM allowlist; `file://` (Electron) matched by bundle ID. **Token binding** — session token bound to OS user session; revoked on agent restart; never in `localStorage`. **MFE attribution** — `X-MFE-Id` is informational, not a security boundary.

B.2 · Client-Side Request Scheduler

Lives in the shell's SDK layer. Enforces 4-slot capacity, per-MFE fairness, priority, and starvation prevention — without ever holding state the agent owns.



SCHEDULING POLICY — WEIGHTED DEFICIT ROUND-ROBIN WITH PRIORITY AGING

```
on enqueue(request, mfe, priority):
    q = subqueue[(mfe, priority)]
    request.enqueue_at = now()
    q.push(request)
    pump()

on slot_released():
    pump()

pump():
    while in_flight < 4 and any_queue_nonempty():
        # 1. Age boost: every 5 s queued, virtual_weight += 1
        for r in all_heads():
            r.effective_weight = base_weight(r) + floor((now -
r.enqueue_at)/5000)
```

```

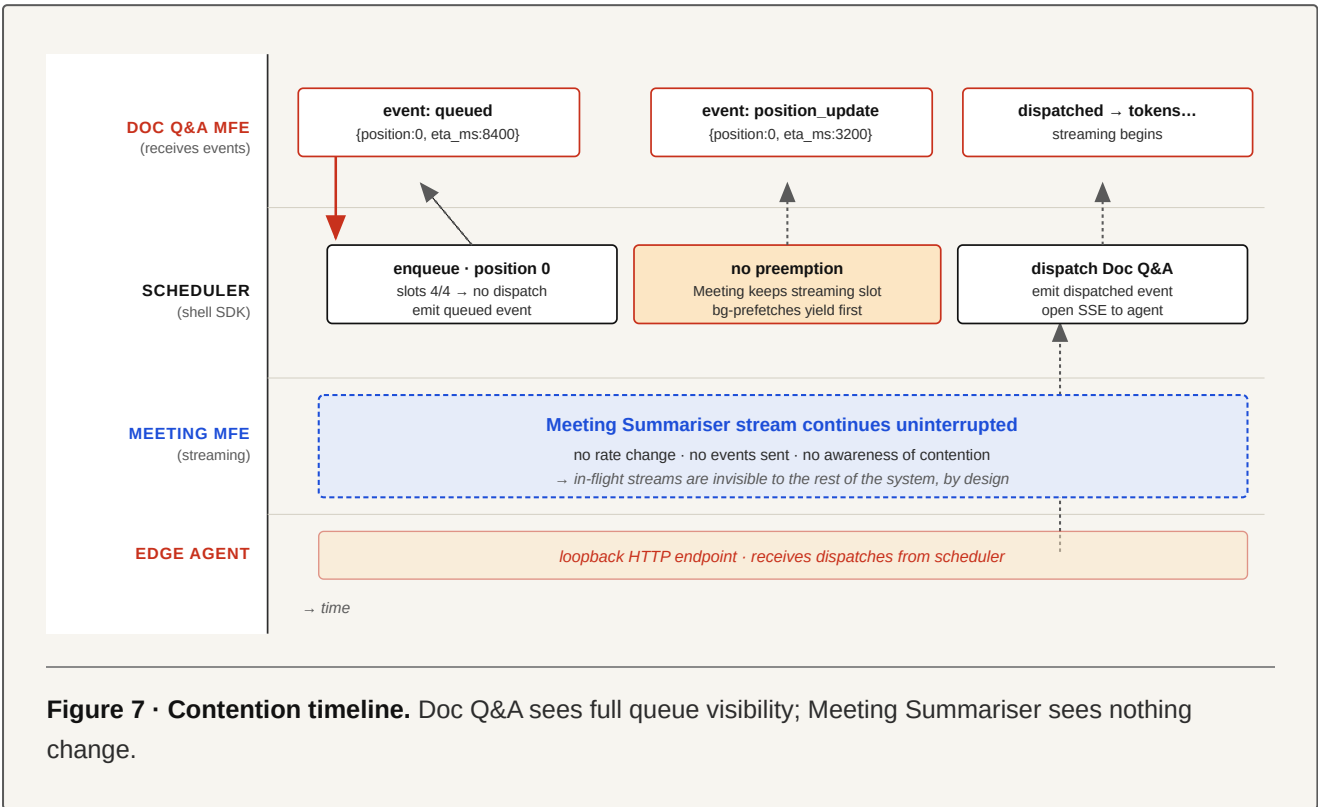
# 2. Per-MFE cap: skip any MFE already at 3 slots
candidates = [q for q in queues if mfe_count(q.mfe) < 3]
# 3. Pick queue with highest deficit_counter (WDRR)
q = argmax(candidates, key=deficit_counter)
r = q.pop()
q.deficit_counter -= r.cost_estimate
dispatch(r)
in_flight += 1

```

CONCERN	MECHANISM
Per-MFE fairness	Per-MFE cap of 3 slots (no MFE can take all 4) + equal base weights for user-priority queues across MFEs.
Priority	User-priority queues have 4× the deficit counter of background queues — user requests pre-empt prefetch in the queue order.
Starvation prevention	Effective weight grows by +1 every 5 s queued. A background request waiting 30 s outranks a fresh user request from the same MFE.
Cancellation	<code>AbortSignal</code> from caller. If queued: O(1) removal from sub-queue. If in-flight: <code>fetch()</code> abort propagates to agent via TCP RST, agent sees client disconnect, frees the slot.
Timeout	Two clocks: <code>queueTimeoutMs</code> (default 10 s) rejects with <code>QUEUE_TIMEOUT</code> ; <code>runTimeoutMs</code> (default 60 s, configurable per request) rejects with <code>RUN_TIMEOUT</code> . Separate codes so the caller knows whether to retry.
Backpressure	Hard queue cap of 32. Beyond that, <code>enqueue()</code> returns a rejected promise with <code>QUEUE_FULL</code> ; the MFE decides whether to drop, debounce, or surface to UI.

B.3 · Streaming + Capacity Contention Scenario

Meeting Summariser is streaming (occupying 1 slot). Three more slots are taken by background prefetches. User opens Document Q&A and submits a query. The agent is at 4/4. This is what every party sees.



SSE EVENT CATALOG

EVENT	PAYLOAD	WHEN FIRED
queued	{position, eta_ms, slots_in_use}	On POST, if dispatch is not immediate.
position_update	{position, eta_ms}	On every queue change ahead of this request. Throttled to 1 per 500 ms.
dispatched	{worker_id, backend, wait_ms}	When a slot is acquired and the agent has accepted the job.
token	{text, index, logprob?}	For streaming responses, on each emitted token.
degraded	See §A.3 / §A.4	If backend swap occurs mid-request.

done	{tokens_emitted, ms_first_token, ms_total, finish_reason}	Successful completion.
error	{code, message, recoverable, resume_token?}	Terminal error or mid-stream crash.
cancelled	{by: "client" "timeout" "scheduler"}	Cancellation acknowledged. Distinguishes cause.

CONTRACT PER CONCERN

CONCERN	CONTRACT
What each MFE receives	Doc Q&A: <code>queued</code> → <code>position_update</code> * → <code>dispatched</code> → <code>token</code> * → <code>done</code> . Meeting Summariser: no change to its stream; nothing leaks about contention.
When updates are delivered	<code>queued</code> within ≤ 50 ms of submit. <code>position_update</code> throttled to ≤ 1 / 500 ms. <code>dispatched</code> within ≤ 30 ms of slot release.
Queue visibility rules	Each MFE sees only its own request's position. ETA is computed from observed token-rate of in-flight streams + average prompt time of queued requests. No cross-MFE info leaks.
Loading semantics	SDK exposes <code>request.state: "queued" "running" "done" "error" "cancelled"</code> as a reactive value the MFE can bind to.
Cancellation	Caller signals via <code>AbortController</code> . If queued: removed in O(1), emit <code>cancelled</code> . If running: <code>fetch()</code> abort → TCP RST → agent frees slot → emit <code>cancelled</code> . Idempotent.
Retry semantics	SDK auto-retries only on <code>recoverable: true</code> errors AND only if <code>X-Idempotency-Key</code> was set by the caller. Otherwise the error is surfaced to the MFE for human decision.

**Streams do not
yield mid-flight**

Once dispatched, a streaming request keeps its slot until **done** .
Preempting would discard KV-cache ($\approx$ seconds of work) and produce worse aggregate latency than waiting. This is the central UX tradeoff of §B.3.

Design Tradeoffs — Summary

Every architectural choice listed below was made by rejecting at least one credible alternative. This page is the audit trail.

DECISION	CHOSE	REJECTED	REASONING
Process count	3 (Supervisor / API / Worker)	2 (Supervisor + combined); 5 (per-backend)	2 fails crash isolation. 5 multiplies model memory beyond budget.
Worker pool size	1 active + 1 warm	1 only; 3-warm pool	1 forces cold-start on every crash. 3-warm wastes ~12 GB for marginal benefit (second crash within recovery window is rare).
IPC mechanism	Named pipes (Win) / UDS (mac)	TCP loopback; gRPC; shared memory	OS-native pipes are fastest and have process-bound lifetimes — pipe dies when worker dies, surfacing the failure immediately.
Wire format	Length-prefixed protobuf	JSON; MessagePack; flatbuffers	Protobuf is fast, schema-evolvable, language-agnostic. JSON allocations would matter at token-streaming rates.
Client transport	HTTP + SSE	WebSocket; gRPC-Web	SSE is unidirectional-server, the streaming shape we need; cancels cleanly via <code>AbortController</code> ; works through enterprise proxies.
MFE integration	Shell-owned singleton SDK	Direct-per-MFE; SharedWorker	Singleton is the only path to honoring the 4-slot global cap with fairness across MFEs.
Scheduling policy	Weighted Deficit Round-Robin + aging	Priority queue; FIFO; lottery	WDRR gives both fairness and priority; aging eliminates starvation. Priority queue alone starves background work indefinitely.

Streaming preemption	Streams never yield mid-flight	Token-budget yielding; head-of-line drop	Yielding discards KV-cache; restart cost > wait cost. Capacity rules are enforced at admission, not in-flight.
Cloud fallback	Opt-in via admin policy only	Automatic on CPU SLA breach	On-device is the product promise. Silent cloud fallback would violate data residency assumptions for enterprises.
Mid-stream crash recovery	Surface as error with <code>resume_token</code> ; client re-prompts	Server-side KV checkpointing	KV checkpointing every N tokens costs ~10–20% of throughput. Crashes are rare; tax should be on the rare path.

Performance & Reliability

Expected latencies



PER-STAGE BUDGET — COLD START

STAGE	BUDGET	NOTES
Supervisor + API spawn	≤ 100 ms	Native binaries, no dependency load.
Worker process spawn	≤ 60 ms	<code>posix_spawn</code> / <code>CreateProcessW</code> .
Model <code>mmap</code> + first faults	1.5 – 2.5 s	Dominant cost. Page-prefault on warm-pool worker brings this to ≈ 0 at handoff.
NPU / ANE context init + graph compile	300 – 800 ms	Cached compiled graph drops to ~80 ms on second start.
First-token generation	120 – 250 ms	Prefill of typical 200-token prompt.

Failure modes & mitigations

FAILURE	DETECTION	MITIGATION	USER IMPACT
Worker SIGSEGV	<code>waitpid</code> / <code>JobObject</code>	Warm-pool promotion (§A.2)	Streaming: ≤ 1 SSE retry event. Non-stream: invisible.
NPU driver removed	<code>QnnGraph</code> error code	Circuit breaker → CPU (§A.3)	4× slower, marked via <code>degraded</code> event.
ANE contention	Core ML error	Metal → Accelerate ladder (§A.4)	2–6× slower, <code>session_scoped:false</code> .

Supervisor crash	Service watchdog	Full agent restart; clients re-handshake	≤ 4 s outage; in-flight requests fail with retryable code.
OOM kill of worker	OS exit code / signal	Same as SIGSEGV path; warn telemetry	Same as crash. Repeated OOM disables KV-cache prefill until restart.
OS sleep / wake	Power notification	Drain in-flight, snapshot warm-pool desc., resume on wake	≤ 2 s reconnect; clients see one <code>retry</code> event.
Disk full (crash dumps)	<code>statvfs</code> at write	5/min rate limit + LRU prune of <code>crashes/</code>	Silent; logged.
Antivirus quarantines worker	Spawn ENOENT	Surface as <code>WORKER_BINARY_MISSING</code> ; UI prompts user to whitelist	Agent unusable until resolved; clear actionable error.
Cascading crash (warm dies during recovery)	Second <code>WORKER_LOST</code> within 1 s	Exponential backoff on spawn; mark agent <code>UNHEALTHY</code> ; refuse new requests with 503	Honest failure; prevents thrashing.
Client disconnect mid-stream	TCP RST / SSE error	Agent cancels job, frees slot; scheduler pumps next	None — clean cleanup.

— Reliability targets — proposed SLOs

- **Crash recovery success rate** ≥ 99.5% within 500 ms (measured: warm-pool worker accepts replayed job).
- **Crash-induced visible error rate** ≤ 0.5% of all in-flight requests during a worker crash event.
- **Steady-state slot utilization fairness**: no MFE's 99th-percentile queue wait exceeds 2× the system mean over a rolling 5-minute window.
- **Cold-to-warm transition**: 95% of agent restarts have a warm worker ready within 3.5 s of API listener up.

Companion Artifacts & Open Questions

— Companion artifacts

GITHUB REPOSITORY

[github.com/\[your-handle\]/sarvam-edge-design](https://github.com/[your-handle]/sarvam-edge-design)

Holds the deeper material that would have bloated this PDF: full edge-case catalog, IPC protocol spec with protobuf schemas, complete API `openapi.yaml`, TypeScript type definitions for every event payload, and prior-art notes (Chromium site isolation, VS Code extension host, Ollama, llama.cpp).

INTERACTIVE SIMULATOR

sarvam-edge-demo.vercel.app

A web app that simulates the runtime described in this document. Three screens: *Process Dashboard* (buttons to crash the worker, OOM-kill, remove the NPU — watch the recovery sequence play out), *Scheduler Visualizer* (fire requests from each MFE at any priority, see WDRR pick winners in real time), and *Contention Scenario* (one-click replay of §B.3 with full SSE event traces visible side by side).

Direct links: [/dashboard](#) · [/scheduler](#) · [/contention](#)

— Open questions I'd want to discuss with the team

1. **Worker pool size at peak load.** The 1+1 design assumes single-user workstations. If a single enterprise workstation has many concurrent MFEs all asking for prefetch, does it make sense to elastically grow the pool (subject to RAM headroom), or stay strict at 1+1 and let the scheduler queue absorb the bursts?
2. **Multi-model future.** What's the roadmap shape — multiple specialized smaller models in parallel, or one larger general model? Affects whether the worker should support hot model-swap (today's design does not) or whether per-model worker pools are introduced.
3. **Cloud fallback policy granularity.** Is the opt-in policy per-tenant, per-user, or per-request? Per-request (via header) gives MFEs the most control but pushes policy enforcement into the agent. Per-tenant simplifies the agent at the cost of MFE flexibility.
4. **Telemetry boundary.** Crash dumps are sensitive (may contain prompt fragments). Default I'd propose: store locally, never auto-upload; offer a manual "share with Sarvam support" flow. Confirming this matches the data-handling stance the team intends.

Headline tradeoffs, the failure modes that matter, and the contracts a team would write code against are all here. The companion repo and demo carry the rest. Thank you for the assignment — the problem is genuinely interesting.